

Curs 2 — Structuri de Date (Gabriel Istrate)

Sinteză: Stive, Cozi, Deque-uri, Mulțimi dinamice, Tabele cu adresare directă, Liste înlănțuite, Skip Lists

Notăție folosită mai jos: 🎯 **Examen** = punct care apare (sau poate apărea) pe grilă, cu trimitere la harta de examene. ⚠️ = avertisment/capcană. 🌱 = exercițiu „de discutat în clasă” pe care l-am rezolvat.

0. Harta cursului (ce e teorie de fundal vs material fierbinte)

Secțiune	Greutate de examen
Motivație (selection sort vs heapsort, „de ce structuri de date”)	conceptual, rar direct
Stive (LIFO, postfix, aplicații)	mare — secvențe imposibile, DFS/Graham, arbore de expresie
Cozi (FIFO, implementare circulară)	mare — BFS folosește coadă
Deque-uri	mediu — BFS modificat cu deque → preordine
Mulțimi dinamice (INSERT/DELETE/SEARCH/MIN/MAX/SUCC/PRED)	fundamental — vocabularul pentru „alege structura”
Tabel cu adresare directă	conceptual — motivează hashing
Liste înlănțuite (cu pointeri!)	mare — pointeri = $n+1$, insert la cap (cod), reverse print
Skip Lists	mare — sortat ascendent, prob. $1/2$, poziție element, numărare pointeri

1. De ce structuri de date? (motivația)

1.1 Exemplul cheie: Selection Sort vs. HEAPSORT

Ideea care motivează tot semestrul — o structură de date bună transformă un algoritm

lent într-unul rapid.

- **Selection Sort:** găsești maximumul din vector, îl muți la final, repeți recursiv.
 - Găsirea maximumului într-un vector: $\Theta(n)$.
 - Recurența: $T(n) = T(n-1) + \Theta(n) \rightarrow \Theta(n^2)$.
- **Gâtul de sticlă (bottleneck):** găsirea maximumului costă $\Theta(n)$.
- **HEAPSORT:** aceeași idee, dar înlocuiești „găsește maximumul” cu o operație $O(1)$ (vârful heap-ului), plus refacerea heap-ului cu HEAPIFY în $\Theta(\log n)$.
 - Rezultat: $\Theta(n \log n)$.

De reținut: identifici operația cea mai costisitoare și frecventă, apoi o accelerezi cu o structură de date potrivită.

1.2 De ce contează: modularitate + performanță

- **Modularitate:** nu construiești o casă din cărămizi de fiecare dată; la fel cu algoritmi. Rezolvi/testezi o problemă o singură dată, corectitudinea se verifică și se actualizează mai ușor.
- **Performanță:** cauți pe Google → nu aștepti 100 de secunde; joci un joc → engine-ul trebuie să acceseze/actualizeze rapid obiectele din câmpul vizual. **Operațiile cele mai frecvente trebuie să fie rapide.** Măsurăm cu $O(n \log n)$, $\Theta(\log n)$ etc.

1.3 Exemplu: operații din cerințe (TCP)

TCP buffer-urile pachete sosite în neregulă. Cerințe → operații:

- buffer-urile un pachet out-of-order,
- scoate (pop) elementele care devin în ordine,
- testează dacă buffer-ul e gol,
- produce primul element lipsă (ACK).
- Vrem performanță $O(1)$ pe operație. → Cerințele dictează ce operații trebuie să fie rapide, deci ce structură alegi.

1.4 Ce ESTE o structură de date (definiție de examen conceptual)

O structură de date este o modalitate de a **organiza și stoca informația** pentru a facilita accesul (sau alte scopuri). Are trei componente:

1. **Date** (ex: un număr complex = două float-uri).
2. **Operații** pe acele date (ex: pentru întregi: +, -, ·).
3. **Garanții de performanță.**

Mai are:

- O **interfață** = proceduri pentru adăugare, ștergere, acces, reorganizare.
- **Date + posibil meta-date** (ex: un heap are vectorul A pentru chei **plus** variabila `A.heap-size` care reține câte elemente sunt în heap).

⚠ „**Concepts, not code**”: modul precis de implementare a operațiilor **NU** face parte din specificația structurii de date. Specifici ce face și *cu ce cost*, nu *cum* e scris codul.

1.5 Structuri de date vs. clase C++

- În C++: clasele = tipuri de date definite de utilizator; obiectele = instanțieri.
 - Obiectele au comportament (funcții membru). Ex: `dog` e o clasă, `Buddy` un obiect, `Buddy.bark()` apelează funcția membru `bark`.
 - Doar funcțiile membru **public** pot fi apelate din afara codului clasei.
-

2. Stive (Stacks) — LIFO

Definiție: organizare secvențială în care **ultimul element inserat e primul scos** — LIFO (*last in, first out*). Exemple: teanc de tăvi/farfurii.

- **Singurul element accesibil:** cel mai recent inserat (vârful).
- **Două operații de bază:**
 - `PUSH` (insert),
 - `POP` (citește și șterge vârful).
- Implementare clasică: vector + variabilă `S.top` (indexul vârfului). `PUSH` incrementează `top` și scrie; `POP` citește și decrementează.

🎯 **Examen — secvențe imposibile pe stivă** (model Q10, grila 2023 Q11): o secvență de `PUSH/POP` e imposibilă dacă faci **POP dintr-o stivă goală**. „Stivă cu k elemente” în enunț = pornește cu k elemente; numeri `PUSH`-urile (+1) și `POP`-urile (−1) și verifici să nu scazi sub 0.

2.1 Aplicație: evaluarea expresiilor postfixate (Reverse Polish Notation)

- **Infix** (forma uzuală): `term1 op term2`, ex: `5 * (((9+8)*(4*6))+7)`.
- **Postfix** (postfixată): `term1 term2 op`, ex: `5 9 8 + 4 6 * * 7 + *`.
- (Conversia infix → postfix: vezi §11.3.)

Algoritmul de evaluare cu stivă (idee):

- citești câte un caracter;
- dacă e **cifră**, construiești numărul: $x = 10 * x + (cifra)$ (cifra cea mai recentă devine cea mai puțin semnificativă), apoi îl pui pe stivă (**push**);
- dacă e **operator**, scoți ultimele **două** valori (**pop** de două ori), aplici operatorul, pui rezultatul înapoi (**push**);
- la final, **pop** = rezultatul.

⚠ **Atenție la cod (typo în slide-uri):** bucla de citire a unui număr multi-cifră ar trebui să aibă acolade:

```
cpp
while (c >= '0' && c <= '9') {
    x = 10*x + (c - '0');
    cin.get(c);
}
```

Fără acolade, doar prima linie e în buclă și **cin.get(c)** rulează o singură dată după — caracterul nu avansează → buclă infinită. Intenția e parsarea unui număr din mai multe cifre.

⚠ **Operatori necomutativi:** la **+** și ***** ordinea scoaterii nu contează. Dar la **-** și **/** contează: trebuie $b = \text{pop}(); a = \text{pop}(); \text{rezultat} = a - b$ (al doilea scos e operandul stâng). Exemplul din curs folosește doar **+** și *****, deci scapă de problemă.

🔗 **Examen — arbore de expresie** (grila 2023 Q22): evaluarea unui arbore sintactic se face în **postordine** (copiii înaintea părintelui) — exact echivalentul evaluării postfixate. Pentru **root(+)** cu copii **(*)** și **7**, iar **(*)** cu **(+)** și **c**, iar **(+)** cu **a** și **b**: ordinea de evaluare e **a, b, +, c, *, 7, root(+)**.

2.2 Alte aplicații ale stivei

- **Eliminarea recursivității** (recursion removal).
- **Inversarea lucrurilor** (reversing).
- **Apel/return de proceduri:** la apel salvezi starea curentă pe stivă; la return o refaci făcând pop. La return te întorci la cea mai recent activă procedură (potrivire de simboluri, ca parantezele).
- **Limbaje formale:** automate pushdown (PDA).

🔗 **Examen — cine folosește stivă:** DFS (Depth-First Search) și Graham's Scan folosesc stivă. (BFS folosește coadă — vezi §3.)

3. Cozi (Queues) — FIFO

Definiție: containerul ubicuu „first-in, first-out” (FIFO). Primul intrat, primul ieșit.

Interfață:

- `Enqueue(Q, x)` — adaugă x la **coada** (spatele) cozii;
- `Dequeue(Q)` — extrage elementul din **capul** cozii.

Implementare (vector circular de lungime fixă):

- `Q.length` = capacitatea (max atâtea elemente); a depăși → eroare „overflow”.
- `Q.head` = poziția capului cozii (primul element).
- `Q.tail` = prima poziție **liberă** din spate.
- Flag-uri: `Q.queue-full`, `Q.queue-empty`.
- **Wraparound:** când head/tail ajunge la `Q.length`, se resetează la 1 (vectorul e tratat circular).

Enqueue (idee): dacă plin → overflow; altfel scrie `Q[Q.tail] = x`, avansează tail (sau wrap la 1), iar dacă tail ajunge să egaleze head → coada e plină.

Dequeue (idee): dacă gol → underflow; altfel `x = Q[Q.head]`, avansează head (sau wrap la 1), iar dacă head ajunge să egaleze tail → coada e goală; returnează x .

Aplicații: scheduling (disc, CPU), gestionarea congestiei de către sistemul de operare, și 🌀

BFS (Breadth-First Search) — *cozile sunt structura de bază a BFS-ului*. (model Q11 din harta de examene: „cine folosește coadă” → BFS.)

4. Deque-uri (cozi cu două capete)

Ca o coadă, dar poți face enqueue/dequeue la **ambele** capete. Practic, extinzi codul cozii cu încă două proceduri. Scorecard similar cozilor (tot $O(1)$).

✳ **Exercițiu „do it!” din curs (fără rezolvare în slide-uri) — rezolvat:** Pe lângă `Enqueue` (la spate) și `Dequeue` (din față) ale cozii, deque-ul are nevoie de încă două operații. Folosind aceeași convenție (vector 1-indexat, `head` = primul element, `tail` = prima poziție liberă din spate):

Enqueue-Front(Q, x) — adaugă la capul din față:

```

if Q.queue-full: error "overflow"
else:
    if Q.head == 1: Q.head = Q.length      // wrap înapoi
    else:          Q.head = Q.head - 1
    Q[Q.head] = x
    if Q.head == Q.tail: Q.queue-full = true
    Q.queue-empty = false

```

(Mut capul cu o poziție înapoi, cu wraparound, și scriu acolo.)

Deque-Rear(Q) — scoate din spate:

```

if Q.queue-empty: error "underflow"
else:
    if Q.tail == 1: Q.tail = Q.length      // wrap înapoi
    else:          Q.tail = Q.tail - 1
    x = Q[Q.tail]
    if Q.tail == Q.head: Q.queue-empty = true
    Q.queue-full = false
    return x

```

(Ultimul element ocupat e la $\boxed{\text{tail} - 1}$ cu wraparound; dau tail înapoi și returnez ce era acolo.)

🌀 **Examen — BFS modificat cu deque** (model Q14): dacă în loc de coadă folosești un **deque** și adaugi vecinii **la vârful** (la fața) deque-ului, deque-ul **simulează o stivă** → parcurgerea devine echivalentă cu **preordine** (în funcție de ordinea în care adaugi copilul stâng/drept). Cheia: ultimul adăugat la vârf e primul scos = comportament LIFO.

5. Scorecard: Stive & Cozi

Operație	Complexitate
Stack-Empty	$O(1)$ ✓
Push	$O(1)$ ✓
Pop	$O(1)$ ✓
Enqueue	$O(1)$ ✓
Deque	$O(1)$ ✓

Operație	Complexitate
Restricție	doar ordine LIFO/FIFO ✗

Toate operațiile sunt $O(1)$ — prețul e flexibilitatea: accesezi doar într-o ordine fixă.

6. Mulțimi dinamice (tema centrală a semestrului)

Asta e cadrul mental sub care intră aproape toate structurile din materie. **Reține lista de operații** — întrebările de tip „vrem operațiile X în $O(\log n)$, ce structură alegem?” se bazează exact pe ea.

Problema mare a semestrului: reprezentăm o mulțime S ale cărei elemente variază în timp. Vrem unele din operațiile:

- `INSERT( $S, x$ )`
- `DELETE( $S, x$ )`
- `SEARCH( $S, x$ )` — rezultat DA/NU (mai bine: un handle către x dacă e găsit)
- `MIN( $S$ )`, `MAX( $S$ )`
- `SUCC( $S, x$ )` (succesor), `PRED( $S, x$ )` (predecesor)

Exemple deja văzute:

- **Stive** = mulțimi dinamice cu ordine LIFO.
- **Cozi** = mulțimi dinamice cu ordine FIFO.

6.1 Dicționar

Un **dicționar** = structură abstractă care reprezintă o mulțime de elemente (chei) — o mulțime dinamică. Interfața generică:

- `Insert( $D, k$ )` — adaugă cheia k ;
- `Delete( $D, k$ )` — șterge cheia k ;
- `Search( $D, k$ )` — spune dacă D conține k .

Implementări concrete: multe (vom vedea **tabele de dispersie / hash tables**).

 **Examen:** întrebările „cea mai simplă structură de la care pornim pentru operațiile $\{\text{INSERT, DELETE, SEARCH, AVERAGE / median / count în } [a,b]\}$ în $O(\log n)$ ” (din harta de examene) folosesc exact acest vocabular. De obicei răspunsul e un BST echilibrat augmentat (AVL/RB) sau un order-statistics tree.

7. Tabel cu adresare directă (Direct-Address Table)

Cea mai simplă implementare de dicționar.

- Universul de chei: $U = \{1, 2, \dots, M\}$.
- Implementare: un vector T de dimensiune M ; **fiecare cheie are propria poziție în T .**
- Operații:
 - $\text{Direct-Address-Insert}(T, k): T[k] = \text{true}$
 - $\text{Direct-Address-Delete}(T, k): T[k] = \text{false}$
 - $\text{Direct-Address-Search}(T, k): \text{return } T[k]$

Complexitate: toate operațiile sunt $O(1)$ ✓.

⚠ **Atunci de ce nu implementăm orice mulțime așa?** (slide-ul pune întrebarea — răspunsul e dat tot în curs:)

- **Spațiul e $\Theta(|U|)$ $\times$ $|U|$** (universul cheilor) e de regulă uriaș, iar mulțimea reprezentată e mult mai mică decât $|U|$. Deci un direct-address table **risipește enorm de multă memorie**.
- Vrem beneficiile lui (acces $O(1)$) dar cu un tabel de dimensiune rezonabilă → **asta motivează tabelele de dispersie (hashing)**, subiectul următor.

Operație	Complexitate
INSERT	$O(1)$ ✓
DELETE	$O(1)$ ✓
SEARCH	$O(1)$ ✓
MEMORIE	$\Theta(M)$ $\times$

8. Liste înlănțuite (Linked Lists)

⚠ **Cel mai important avertisment al cursului:** „Implementarea cu pointeri: nu treci materia dacă nu o știi!” — și exact asta a fost cerută la examenul de toamnă (scrie o funcție de inserare la începutul unei liste dublu înlănțuite). **Învață codul pe de rost și înțelege-l.**

Interfață:

- `List-Insert(L, x)` — adaugă x la începutul listei;
- `List-Delete(L, x)` — șterge x ;
- `List-Search(L, k)` — găsește elementul cu cheia k .

Implementare (dublu înlănțuită): fiecare element x are:

- `x.prev` și `x.next` (legături către elementul anterior/următor),
- `x.key` (cheia).

8.1 Pseudocod (varianta cu `head` și `NIL`)

```
List-Init(L):
    L.head = NIL

List-Insert(L, x):           // inserare la cap
    1  x.next = L.head
    2  if L.head ≠ NIL
    3      L.head.prev = x
    4  L.head = x
    5  x.prev = NIL

List-Search(L, k):
    1  x = L.head
    2  while x ≠ NIL and x.key ≠ k
    3      x = x.next
    4  return x

List-Delete(L, x):           // x = pointer către nodul de șters
    1  if x.prev ≠ NIL
    2      x.prev.next = x.next
    3  else L.head = x.next
    4  if x.next ≠ NIL
    5      x.next.prev = x.prev
```

Înțelegere: la inserare legi noul nod în fața capului și actualizezi cele patru pointere (next-ul noului, prev-ul vechiului cap, head-ul listei, prev-ul noului = NIL). La ștergere „sari peste” x reconectând vecinii lui; tratezi separat cazul în care x e capul (`x.prev == NIL`).

8.2 Varianta cu santinelă (`L.nil`)

În loc de NIL, un element fictiv (dummy) `L.nil`. **Simplifică LIST-DELETE** (elimini cazurile speciale cu NIL), dar **costă memorie** $\times$.

```

List-Init(L):
    L.nil.prev = L.nil
    L.nil.next = L.nil

List-Insert(L, x):
    x.next = L.nil.next
    L.nil.next.prev = x
    L.nil.next = x
    x.prev = L.nil

```

Căutarea se oprește când ajunge înapoi la `L.nil` în loc de NIL.

8.3 Liste circulare

Folosesc santinela `L.nil` drept „cap” — ultimul nod pointează înapoi la santinelă, formând un ciclu. Aceeași logică de inserare/ștergere/căutare.

8.4 Scorecard

Operație	Complexitate
List-Insert	$O(1)$ ✓
List-Delete (cu pointer către nod)	$O(1)$ ✓
List-Search	$\Theta(n)$ ✗

Observații: inserarea se face implicit la cap; se poate insera și pe poziție arbitrară. Stivele/cozile pot fi reimplementate cu liste înlănțuite.

🔗 Examen — lista înlănțuită:

- **Numărul de pointeri** într-o listă **simplu** înlănțuită cu n noduri, incluzând `head`, este $n + 1$ (n pointeri `next`), dintre care ultimul e NULL dar tot un câmp pointer, plus `head`). (Doc1 Q23.)
- **Inserare la început** într-o listă dublu înlănțuită — codul de mai sus, cerut direct la toamnă.
- **Funcție recursivă** `f(head)` care apelează `f(head->next)` **înainte** de a afișa `head->data` → afișează lista în **ordine inversă**. (model Q24.)

⚠ Nu e în Cormen (vezi Drozdek). Pe examen apare des: proprietăți, numărarea pointerilor, poziția unui element.

Problema: într-o listă înlănțuită, **căutarea e lentă ($\Theta(n)$)** chiar și când elementele sunt sortate (nu ai acces aleator). **Soluția:** liste de elemente ordonate care permit „săritul” peste **elemente** ca să accelereze căutarea. Skip list = variantă de listă ordonată cu mai multe niveluri de pointeri.

9.1 Versiunea idealizată (deterministă)

Ideea „ce ne-am dori”:

- fiecare **al 2-lea** nod pointează cu 2 poziții înainte,
- fiecare **al 4-lea** nod pointează cu 4 poziții înainte,
- fiecare **al 8-lea** nod pointează cu 8 poziții înainte, ș.a.m.d.
- Formal: itemul $2^{(k-1)} \cdot i$ pointează către itemul $2^{(k-1)} \cdot (i+1)$.

Numărul de pointeri diferă de la nod la nod:

- jumătate din noduri au 1 pointer,
- un sfert au 2 pointeri,
- o optime au 4 pointeri, ș.a.m.d.
- → cursul dă cifra $n \cdot \log_2(n) / 2$ pointeri pentru această variantă rigidă.

(Această variantă deterministă corespunde **cazului defavorabil** din scorecard: spațiu $O(n \log n)$. Varianta randomizată reală — mai jos — folosește **$O(n)$ spațiu în medie**, ceea ce e și cifra din scorecard.)

9.2 Algoritmul de căutare

1. Urmează pointerii pe **nivelul cel mai înalt** până găsești un element mai mare sau termini lista.
2. Dacă ai depășit (element mai mare), te întorci la **predecesor** și continui de la el pe **nivelul imediat inferior**.
3. Repeti până găsești elementul, sau ajungi pe primul nivel și dai de un element mai mare / capătul listei.

Intuiție: cobori în „trepte” tot mai fine — sari mult cât poți, încetinești când te apropii de țintă.

⚠ **Problema versiunii rigide:** la inserare/ștergere ar trebui **restructurați pointerii** tuturor nodurilor (ca să păstrezi spațierea exactă) — prea scump. **Soluția: spațiere aleatoare** pe fiecare nivel, în loc de exactă.

- **Invariant:** numărul de noduri pe fiecare nivel rămâne, **în medie**, egal cu cel din versiunea cu spațiere exactă.
- **Principiu:** „dacă parcurgi 10 metri în 10 pași, un pas e în medie un metru.”

9.4 Alegerea nivelului unui nod nou

- **Numerotarea nivelurilor începe de la 0.**
- Un nod nou ajunge: cu probabilitate $1/2$ pe primul nivel, $1/4$ pe al doilea, $1/8$ pe al treilea, etc. (distribuție geometrică — o aruncare de monedă „mai sus?” la fiecare nivel, prob. $1/2$).
- Funcția `chooseLevel`: generează un număr aleator în $[0,1]$; dacă e în $[0, 1/2]$ → nivel 1, $[1/2, 3/4]$ → nivel 2, etc.
- La **ștergere**: trebuie actualizate **toate** legăturile nodului.

🎯 Examen — proprietăți skip list (model Q9):

- **Elementele sunt sortate crescător** — ADEVĂRAT.
- „Probabilitatea ca un nod să aibă doi pointeri este exact $1/4$ ” — soluția-model o tratează drept **FALSĂ**, fiindcă un nod e promovat la nivelul următor cu probabilitate $1/2$ (deci „are doi pointeri” = ajunge cel puțin pe nivelul 1 = prob. $1/2$ în convenția examenului). ⚠ *Nuanță:* dacă interpretezi strict „exact doi pointeri” = „exact nivelul 1” = $(1/2) \cdot (1/2) = 1/4$ — dar răspunsul așteptat la examen e **$1/2$** , conform convenției profesorului că promovarea per nivel e $1/2$. Reține răspunsul de examen: **$1/2$** .
- „Nivelurile sunt spațiate egal” — FALS (spațiere aleatoare).

🎯 **Examen — poziția unui element** (model Q20, grila 2023 Q20): elementele fiind sortate crescător, „al câtelea nod conține elementul x” = poziția lui x în ordinea sortată. Ex: în $\{1,2,3,5,8,13,21,44\}$, elementul 5 e în al **4-lea** nod, elementul 8 în al **5-lea**.

🎯 **Examen — modificarea pointerilor la inserare** (Doc1 Q21): la inserarea unui nod cu nivel maxim 2 (niveluri de la 0, deci nodul există pe nivelurile 0,1,2), în general redirecțezi pointerul predecesorului pe fiecare din cele 3 niveluri (până la 3 noduri distincte) **plus** setezi cei 3 pointeri ai nodului nou → în general **4 noduri**.

9.5 Indexing (mimează indexarea în $O(\log n)$) — precursor de order-statistics

Putem calcula al i-lea element **mai rapid decât** $O(i)$ dacă reținem „**dimensiunile pașilor**”

(step sizes) pe fiecare pointer:

- pornești pe nivelul cel mai înalt; dacă un pas e prea mare, te întorci la predecesor și cobori un nivel; continui până ajungi la elementul i .
- Actualizezi step-sizes la inserare/ștergere (ușor cu liste **dublu** înlănțuite):
 - la ștergere: `pred[i].size += deleted.size` pe toate nivelurile i ;
 - la inserare: ții evidența predecesorilor și a indexului secvenței inserate.

🌀 Acest „step size” e fix ideea de **câmp de dimensiune (size/tree-size)** care apare la **order-statistics trees** (vezi harta de examene: SELECT al k -lea element, $\text{query}(a,b)$, median). Skip list-ul cu step-sizes e analogul probabilist.

🌀 **Examen — numărarea pointerilor într-un skip list modificat** (model Q16): variantă dublu înlănțuită, 4 niveluri $\rightarrow$ numărul mediu total de pointeri e $\Theta(n)$, calculat prin **sumă geometrică pe niveluri** ($\approx 2(n+1)$ pe primul nivel, apoi $\sim$ se înjumătățește la fiecare nivel superior). Metoda: scrii suma pointerilor per nivel și observi că converge la $\Theta(n)$.

9.6 Scorecard

Metodă	Mediu	Caz defavorabil
SPACE	$O(n)$	$O(n \log n)$
SEARCH	$O(\log n)$	$O(n)$
INSERT	$O(\log n)$	$O(n)$
DELETE	$O(\log n)$	$O(n)$

- ✓ Foarte practic.
- ✗ Probabilist (cazul defavorabil rămâne prost).
- ✗ Nu e foarte ușor de implementat.
- Comparat cu ce? Cu **arborii binari de căutare** (vom învăța mai târziu). Skip list-urile au **comportament de cache mai prost**.

Aplicații reale (cultură generală, nu de memorat): Redis (ordered sets), LevelDB, MemSQL, Lucene, ConcurrentSkipListMap (Java), kernel-ul Linux (scheduler MuQSS) etc.

10. Cheat-sheet — fapte și complexități de reținut

Concept	De reținut
Stivă	LIFO; push/pop $O(1)$; secvență imposibilă = POP din gol
Coadă	FIFO; enqueue/dequeue $O(1)$; implementare circulară cu head/tail/full/empty
Deque	ambele capete, $O(1)$; BFS cu deque (adăugare la vârf) → preordine
Evaluare postfix	cu stivă; arbore de expresie = postordine
Cine folosește stivă	DFS, Graham's Scan
Cine folosește coadă	BFS
Mulțime dinamică	INSERT, DELETE, SEARCH, MIN, MAX, SUCC, PRED
Direct-address table	toate $O(1)$, dar **spațiu $\Theta()$
Listă înlănțuită	Insert $O(1)$, Delete-cu-pointer $O(1)$, Search $\Theta(n)$
Pointeri listă simplă (n noduri, +head)	$n + 1$
Listă cu santinelă	simplifică Delete, costă memorie
Skip list — sortare	crescător
Skip list — promovare nivel	probabilitate $1/2$ per nivel (niveluri de la 0)
Skip list — pointeri (idealizat)	$n \cdot \log_2(n)/2$
Skip list — complexități	Search/Insert/Delete: $O(\log n)$ mediu, $O(n)$ worst; Space $O(n)$ mediu
Skip list — poziția elementului	= rangul lui în ordinea sortată

11. Întrebări/exerciții „de discutat în clasă” — rezolvate

11.1 ✖ „Deque: do it!” (cele două proceduri lipsă)

Rezolvat în §4: `Enqueue-Front` (mută head înapoi cu wraparound și scrie) și `Dequeue-Rear` (mută tail înapoi cu wraparound și returnează). Vezi pseudocodul de acolo.

11.2 ✖ „De ce nu implementăm orice mulțime cu un direct-address table?”

Rezolvat în §7: pentru că **spațiul e** $\Theta(|U|)$, iar $|U|$ (universul cheilor) e de obicei uriaș față de mulțimea efectiv stocată → risipă masivă de memorie. Soluția practică e hashing-ul (tabel de dimensiune rezonabilă cu o funcție hash).

11.3 ✖ „Cum convertim infix → postfix?” (cursul zice „later!”)

Metoda standard e **shunting-yard** (Dijkstra), tot cu o **stivă** (de operatori):

- parcurgi expresia infixată stânga→dreapta;
- **operand** → îl scrii direct la ieșire (postfix);
- **operator** → cât timp în vârful stivei e un operator cu prioritate $\geq$ (ținând cont de asociativitate), îl scoți la ieșire; apoi pui operatorul curent pe stivă;
- **paranteză deschisă** `(` → pe stivă;
- **paranteză închisă** `)` → scoți operatori la ieșire până dai de `(`, pe care o arunci;
- la final, scoți tot ce a rămas pe stivă la ieșire.

Exemplu: `5 * ((9+8)*(4*6))+7` → `5 9 8 + 4 6 * * 7 + *` (exact forma postfixată din curs).

11.4 ✖ Nuanța „probabilitatea de doi pointeri” (skip list)

Discutată în §9.4: răspunsul de examen e $1/2$ (promovare per nivel), deși interpretarea strictă „exact doi pointeri” ar da $1/4$. Mergi pe convenția profesorului: $1/2$.

Bottom line pentru examen din acest curs: (1) stăpânește stiva — secvențe imposibile + postfix/arbore de expresie în postordine + DFS/Graham; (2) coada → BFS, deque → BFS-ul devine preordine; (3) memorează codul de inserare/ștergere în liste dublu înălțuite (avertisment direct: „nu treci fără el”) și fapta $n+1$ pointeri; (4) skip list: sortat crescător, promovare $1/2$ per nivel, poziția = rang, numărarea pointerilor prin sumă geometrică ($\Theta(n)$); (5) reține lista operațiilor de mulțime dinamică — e cadrul tuturor întrebărilor „alege structura”. Direct-address table-ul e doar puntea conceptuală către hashing, subiectul din cursul următor.